HACKATHON SUBMISSION

Hackovium

VibeCraft Edition

DealCart

Intelligent e-commerce price tracking with algorithmic fake sale detection.

THE TEAM

AM
Ankush Mahato
Team Leader

UD
Upasona Das
Member

AY
Akash Yadav
Member

AS
Aditya Srivastava
Member

THEME

01

E-Commerce & Marketplaces

DEFINING THE THEME

India's e-commerce market is projected to reach **$200 billion by 2027**. With millions of consumers shopping across Amazon, Flipkart, and other platforms, price transparency has become a critical need. This theme challenges us to build tools that empower consumers in the digital marketplace — helping them make informed decisions, avoid predatory pricing, and get genuine value for their money.

PROBLEM STATEMENT

The Hidden Cost of Online Shopping

Dynamic Pricing:	Frequent unannounced price fluctuations mask true value.
Fake Discounts:	Sellers artificially inflate base prices prior to major sales events.
Siloed Platforms:	Manual cross-platform price checking is inefficient.
Missed Drops:	Lack of real-time monitoring leads to missed purchasing windows.

67%
of Indian consumers encounter "fake discounts"

₹2,400
average overpayment per consumer/year

3.2x
daily price changes per product

OUR SOLUTION

Introducing DealCart

DealCart provides automated price tracking via JSON-LD extraction, historical data visualization, and V-curve algorithmic analysis to flag artificial discounts across major e-commerce platforms.

Paste URL — Amazon / Flipkart → **Smart Scraper** — JSON-LD + Cheerio → **Supabase DB** — PostgreSQL + RLS → **AI Analysis** — Insights Engine → **Dashboard** — Charts + Alerts

HOW WE BUILT IT

The Engineering Behind DealCart

INTELLIGENT SCRAPING

We use a dual-strategy scraping engine — first attempting structured JSON-LD extraction (fastest, most reliable), then falling back to HTML selectors with Cheerio. Built-in retry logic with exponential backoff ensures resilience against rate-limiting.

FAKE SALE ALGORITHM

Our V-curve detection algorithm analyzes the last 10 price data points, identifies artificial hikes (≥15%), quantifies the subsequent "discount", and flags products where the current price remains near or above the pre-hike baseline.

DEAL SCORING ENGINE

A multi-factor scoring system (0-100) evaluates each product against its all-time low, average price, recent trends, and volatility to generate actionable buy/wait recommendations.

ROW LEVEL SECURITY

Every database table is protected by Supabase RLS policies. Users can only view, insert, update, or delete their own data — ensuring complete multi-tenant isolation with zero middleware overhead.

KEY FEATURES

03

What Makes DealCart Powerful

Real-Time Price Tracking

Add any Amazon or Flipkart product URL and instantly begin tracking its price history. Visual sparkline charts and interactive Recharts graphs make trends clear at a glance.

Fake Sale Detection

Our proprietary V-curve algorithm detects artificial price hikes followed by deceptive "discounts." Products are flagged with a prominent warning badge to protect consumers.

Smart Deal Scoring

Each product receives a 0-100 Deal Score based on historical lows, average price, trend direction, and volatility — giving you a clear "Buy Now" or "Wait" signal.

Budget Alerts

Set custom min/max budget ranges per product. When the price enters your budget, DealCart sends an instant email notification so you never miss a deal.

Social Deal Sharing

Found a great deal? Share it instantly with friends via Twitter, WhatsApp, or simply copy a pre-formatted deal summary with price and savings breakdown.

AI-Powered Insights

Smart recommendations analyze volatility, savings vs. peak/average, price trends, and seasonality to give you contextual buying advice for every product.

Responsive Dark Mode UI

A premium glassmorphism design with dark mode, animated buttons, scrolling price ticker, 3D card tilts, and micro-interactions for a seamless experience on every device.

Email Notifications

Beautiful HTML email templates for budget alerts, price drops, and welcome messages — designed to match the DealCart brand identity with actionable "Buy Now" CTAs.

TECHNOLOGY

04

Tech Stack & Architecture

Next.js 16
App Router, SSR, API Routes

TypeScript
Type-safe codebase

Supabase
Auth, PostgreSQL, RLS

Recharts
Interactive price charts

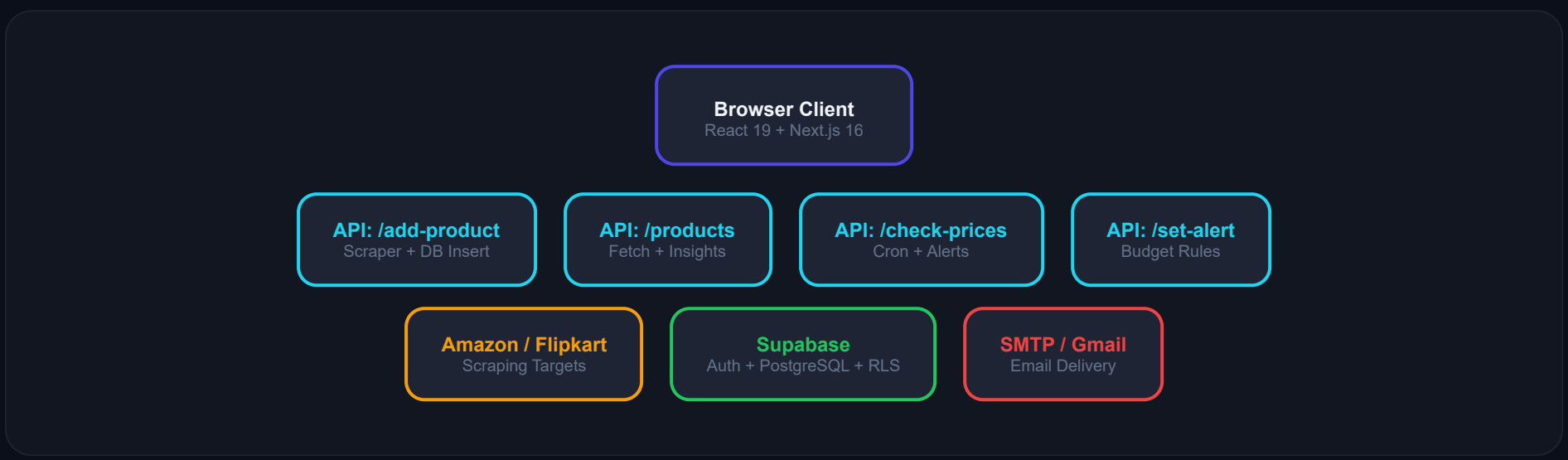
Cheerio + Axios
Web scraping engine

Tailwind CSS 4
Utility-first styling

Nodemailer
Email notifications

React 19
Latest concurrent features

System Architecture



Database Architecture

ENTITY RELATIONSHIP DIAGRAM

profiles

id	UUID
email	TEXT
telegram_chat_id	TEXT
notification_type	TEXT
created_at	TIMESTAMPTZ

products

id	UUID
user_id	UUID FK
title	TEXT
url	TEXT
image_url	TEXT
platform	TEXT
current_price	NUMERIC
original_price	NUMERIC
rating	TEXT
availability	TEXT

price_history

id	UUID
product_id	UUID FK
price	NUMERIC
checked_at	TIMESTAMPTZ

alerts

id	UUID
product_id	UUID FK
user_id	UUID FK
min_price	NUMERIC
max_price	NUMERIC
alert_sent	BOOLEAN

ROW LEVEL SECURITY (RLS) POLICIES

User Isolation
auth.uid() = user_id on all tables

Cascading Access
Price history via product ownership

Service Role
Cron jobs bypass for price updates

CODEBASE

06

Project Structure

```
DealCart-Project/ ├── src/ (Main Application Code) | ├── app/ (User Screens & Screens) | | ├── page.tsx
← The welcome landing page | | ├── login/ & signup/ ← Where users create their accounts | | ├──
dashboard/ ← Where users view tracked items | | | ├── page.tsx ← Grid showing all saved items | | | └──
product/[id]/ ← Price history graphs and alerts config | | └── api/ (Backend Server Automation) | | ├──
add-product/ ← Connects to Amazon/Flipkart to read prices | | ├── products/ ← Fetches and deletes items
for the user | | ├── check-prices/ ← Background job that runs automatically | | └── set-alert/ ← Saves
the user's budget preferences | ├── components/ (Reusable Design Parts) | | ├── layout/ ← The top
navigation and side menus | | └── price-ticker.tsx ← Rolling text that shows live updates | └── lib/
(Core Brain of the App) | ├── scraper/ ← The tool that reads website data automatically | ├── insights/
← The algorithm that detects "Fake Sales" | └── notifications/ ← Tool that sends emails to customers ├──
supabase/ (Database Settings) | └── migration.sql ← Blueprint that creates our storage tables └──
package.json ← Master list of all tools used in the project
```

App Screens (User Pages)

PAGE	/	Welcome screen
PAGE	/login & /signup	Account access
PAGE	/dashboard	Main tracker grid
PAGE	/dashboard/product/[id]	View graphs and alerts

Backend Actions (Server API)

POST	/api/add-product	Find & save new item
GET	/api/products	Get user's save list
DEL	/api/products/[id]	Remove an item
GET	/api/check-prices	Daily automatic check

USER WORKFLOW

How a User Experiences DealCart

1. **Sign Up** — Create account via Supabase Auth
2. **Paste URL** — Amazon / Flipkart product link
3. **Auto Scrape** — Title, price, image extracted
4. **Track & Analyze** — Charts, scores, insights
5. **Get Alerted** — Email when price drops

FAKE SALE DETECTION ALGORITHM

V-Curve Pattern Analysis

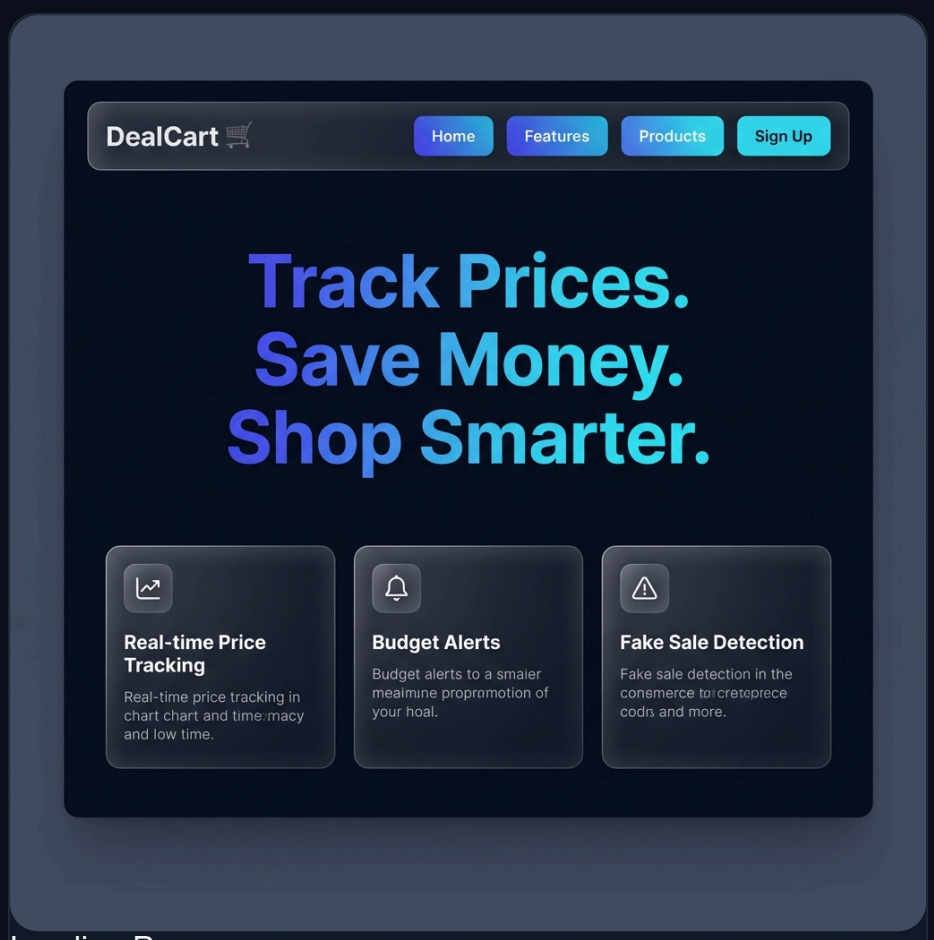
Detection Criteria

1. Minimum **5 data points** required
2. Price hike ≥ **15%** from baseline
3. Subsequent drop ≥ **10%** from peak
4. Current price still **near baseline**
5. Pattern must form **V-curve shape**

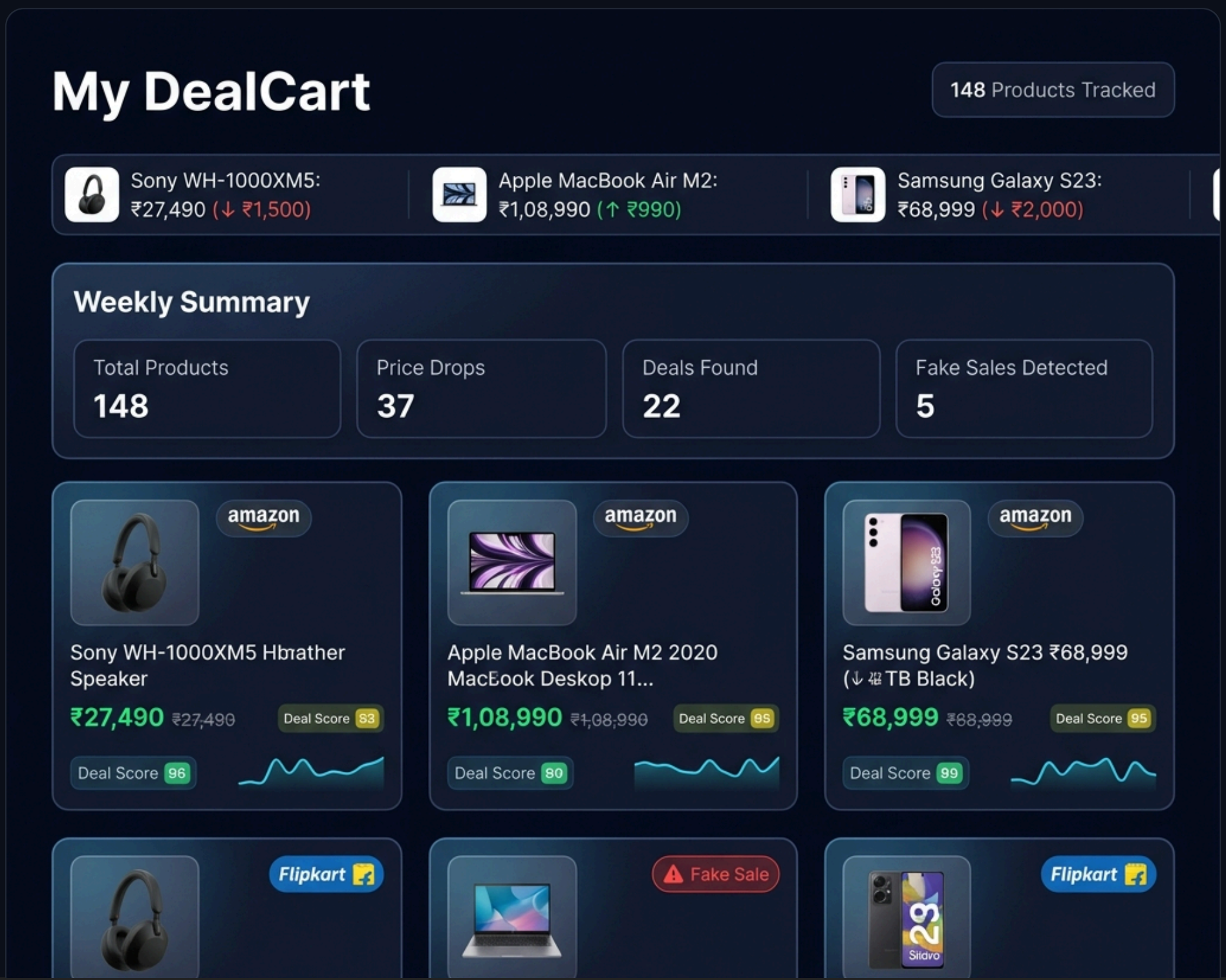
Result: Products matching this pattern are flagged with a FAKE SALE badge on the dashboard.

UI SHOWCASE

App Screenshots



Landing Page



Product Detail Page

Interactive Recharts price history graph, AI insights panel, deal score, savings analysis, and budget alert configuration

Authentication Pages

Glassmorphism login/signup forms with animated gradient buttons, Supabase Auth integration

UI HIGHLIGHTS

Full Dark Mode
Consistent dark theme across every page

Glassmorphism
Frosted glass cards with subtle borders

Micro-Animations
Hover effects, 3D tilts, animated buttons

09

ENTERPRISE READINESS

Scalability & Performance

BUILT FOR HIGH WORKLOADS

DealCart is architected to scale seamlessly as user count grows and the tracking volume increases. Using cloud-native tooling and decoupled asynchronous processing, it maintains high performance even under heavy loads.

Edge Network Caching

By leveraging Next.js on Vercel, static assets and dynamic routes utilize intelligent CDN caching. Users get single-digit millisecond latency regardless of geographic location.

Connection Pooling

To prevent database overload during traffic spikes, Supabase utilizes PgBouncer for efficient PostgreSQL connection pooling, allowing thousands of concurrent queries without server strain.

Decoupled Cron Jobs

Price checking operations run asynchronously via Vercel Cron. Large batches of products are processed in chunks over serverless functions, preventing UI blocking and timeout errors.

Exponential Backoff

The scraper engine includes resilient error parsing and retry constraints. If Amazon/Flipkart imposes momentary rate limits, DealCart backs off exponentially to prevent permanent IP blocks.

99.9%
Uptime via Serverless

< 10ms
DB Query Latency

WHAT'S NEXT

Future Improvements

ML Price Prediction

Implement predictive models using historical price volatility metrics to forecast optimal buying windows over 30-day periods.

Browser Extension

Develop a Chromium extension for real-time overlay alerts, automatically comparing active product pages against DealCart database.

Distributed Scraper Net

Deploy rotating proxy infrastructure and headless browser clusters to bypass advanced anti-bot protections and scale tracking.

Seller Integrity Scoring

Aggregate historical pricing manipulations to assign quantitative trust ratings to specific third-party e-commerce sellers.

Cross-Platform Matching

Utilize AI vision and NLP algorithms to automatically link identical merchandise across Amazon, Myntra, and Flipkart.

API as a Service (SaaS)

Expose our clean JSON-LD extraction engine and V-curve detection logic as a paid enterprise API for retail analysts.

Thank You!

DealCart | Track Smarter. Buy Better.

Ankush Mahato · Upasona Das · Akash Yadav · Aditya Srivastava